

CI/CD Pipeline GitHub Actions

SHINY ZERUBBA RAJIVE S

Step 1: Created GitHub Repository

- Opened GitHub
- Created a new repository for CI/CD practice
- Example repo name:

ci-cd_practice1

Step 2: Created README File

Inside the empty repository:

- Clicked:

Add file → Create new file

- Created:

README.md

- Added sample project content

Purpose:

- Makes repository non-empty
- Gives project description
- Standard first file in GitHub projects

Step 3: Opened GitHub Actions

- Clicked:

Actions

Purpose:

- GitHub Actions is used for CI/CD automation

Step 4: Created Workflow YAML File

- Chose:

set up a workflow yourself

GitHub automatically created:

.github/workflows/main.yml

Purpose:

- YAML file defines CI/CD pipeline steps

Step 5: Added Basic CI Pipeline Code

Added workflow YAML:

```
name: Python CI Pipeline
on:
  push:
    branches:
      - main
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout Code
        uses: actions/checkout@v4
      - name: Setup Python
        uses: actions/setup-python@v5
        with:
          python-version: "3.11"
      - name: Verify Python
        run: python --version
```

Purpose:

- Trigger pipeline whenever code is pushed
- Create virtual machine
- Install Python
- Verify Python installation

Step 6: Committed Workflow File

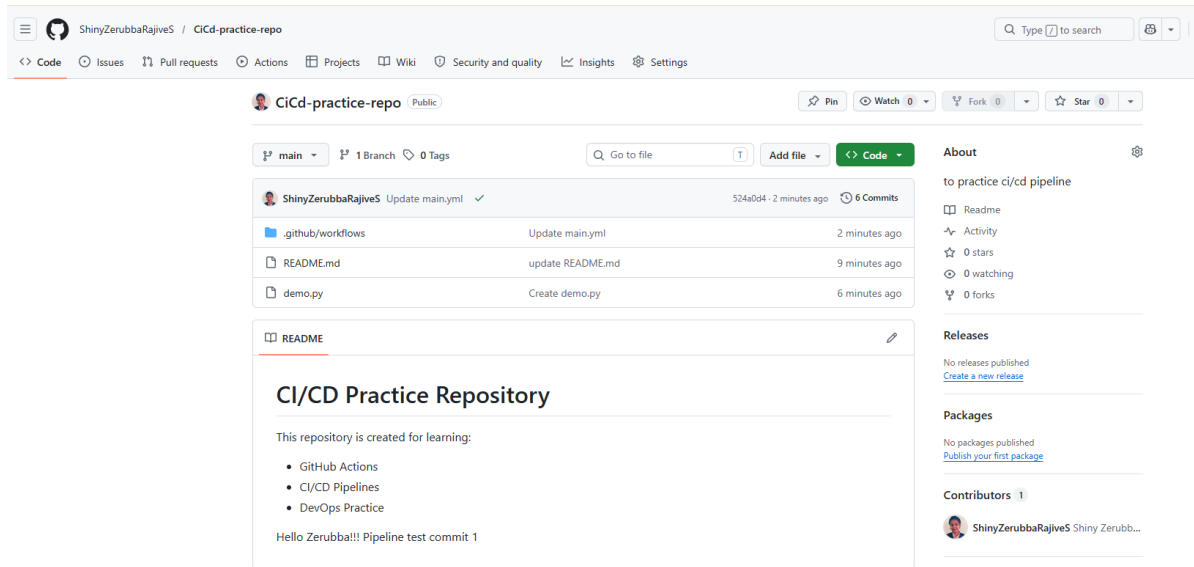
- Clicked:

Commit changes

Purpose:

- Saves YAML file into repository

- Automatically triggers GitHub Actions pipeline



Step 7: Observed Pipeline Execution

Opened:

Actions → Python CI Pipeline

Observed workflow stages:

- Checkout Code
- Setup Python
- Verify Python

Purpose:

- View CI/CD execution logs
- Monitor build success/failure

Step 8: Created Python Application File

Created:

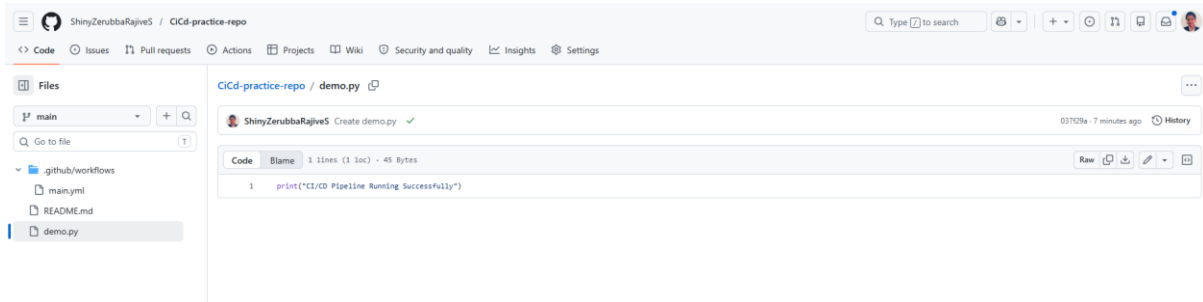
app.py

Content:

```
print("CI/CD Pipeline Running Successfully")
```

Purpose:

- Simple Python application for pipeline execution testing



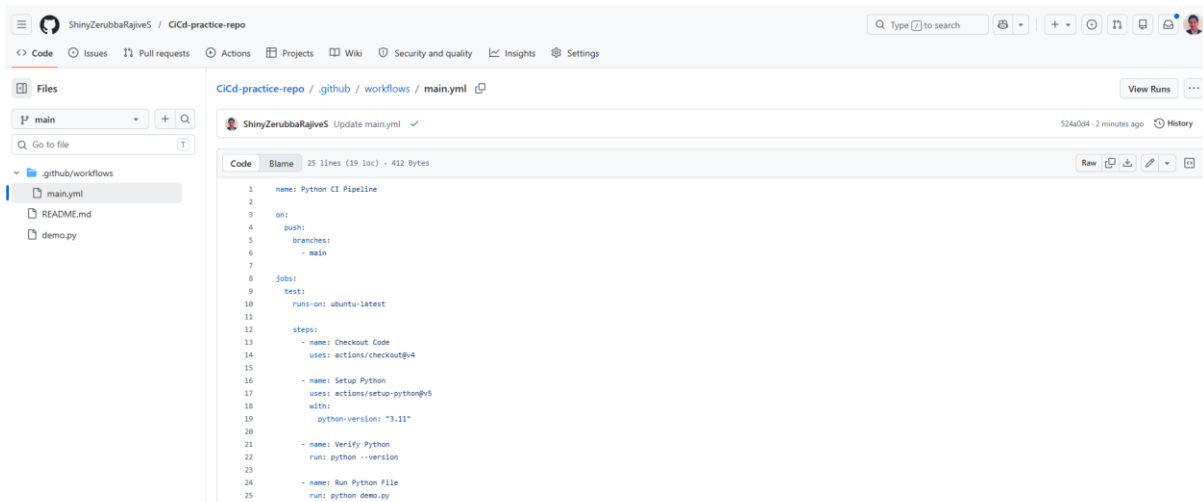
Step 9: Updated YAML to Execute Python File

Added:

- name: Run Python File
- run: python app.py

Purpose:

- Execute application during pipeline run
- Simulate real project execution

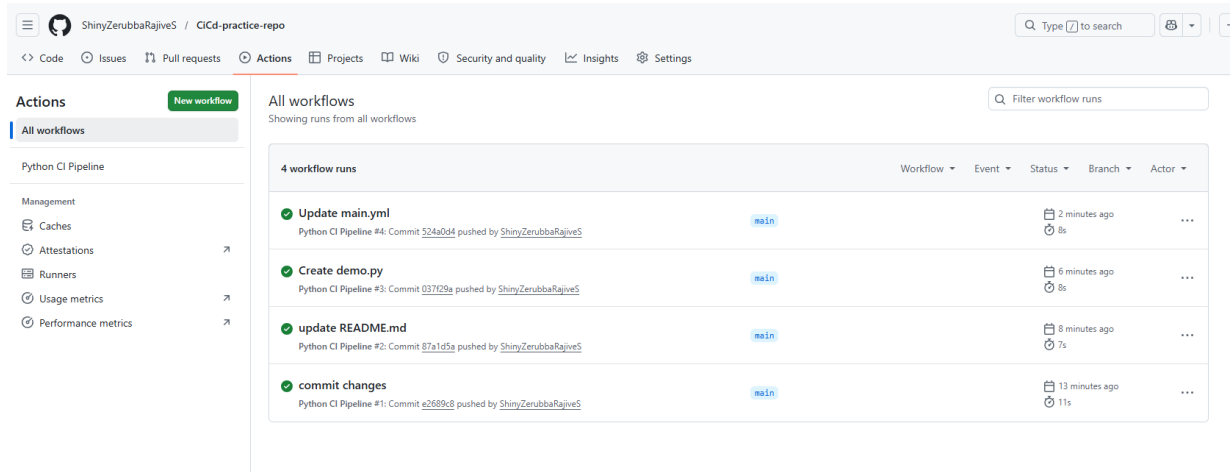


Step 10: Triggered Pipeline Again

- Modified/committed files
- GitHub automatically triggered pipeline again

Purpose:

- Demonstrates Continuous Integration behavior
- Every code push triggers automation



Step 11: Viewed Execution Output

Opened:

Actions → Workflow Run → test → Run Python File

Observed output:

CI/CD Pipeline Running Successfully

Purpose:

- View runtime logs
- Validate application execution in pipeline environment

